

Summary of LLM Agents: Agent Planning Ability

Aishee Mondal (aisheem2), Akshat Bhat (akshatb5), Aniket Tripathy (anikett2)

Problem and Motivation

Large Language Models (LLMs) have struggled with long-horizon problem-solving and complex decision-making, despite their strong reasoning capabilities. Existing approaches like Chain-of-Thought and ReAct have attempted to improve reasoning by breaking problems into sequential steps and incorporating environmental interactions [1,2]. However, they exhibit two critical weaknesses: First, error propagation in long reasoning chains leads to incorrect solutions, as small mistakes in early steps compound over time. Second, these models lack deliberation and structured search, meaning they commit to a single reasoning path without systematically exploring alternative solutions. These limitations make LLMs unreliable for both structured planning tasks, which require generating optimal step-by-step solutions, and adaptive decision-making tasks, which demand evaluating multiple options dynamically.

Together, advancements explored in this discussion such as LLM+P and LATS move beyond the limitations of CoT and ReAct by combining structured, optimal planning (LLM+P) with adaptive, exploratory decision-making (LATS). This synergy enables LLMs to tackle both well-defined planning problems and dynamic, real-world decision-making scenarios, representing a significant step toward more reliable and capable autonomous reasoning systems.

Traditional language models find it challenging to execute real-world multi-step planning, particularly when various constraints such as budget, time, and travel logistics are involved. Previous AI planning efforts have concentrated on structured tasks like robot navigation or theorem proving, but there is a noticeable absence of benchmarks assessing LLMs in dynamic, real-world planning scenarios. To fill this void, TravelPlanner is introduced, systematically evaluating the ability of LLMs to produce feasible and optimal travel itineraries while considering constraints like flight schedules, accommodation availability, activity sequencing, and user preferences. This benchmark is vital for determining whether LLMs can act as dependable assistants in complex, real-world decision-making tasks that go beyond theoretical or static contexts.

Related Works

HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models [5]

HippoRAG proposes a retrieval framework inspired by the hippocampal indexing theory to improve how LLMs store and retrieve knowledge. Unlike traditional retrieval-augmented generation (RAG) approaches, HippoRAG integrates knowledge graphs and Personalized PageRank to mimic the neocortex and hippocampus, enhancing multi-hop reasoning efficiency. This method significantly improves retrieval quality while reducing computational costs, outperforming state-of-the-art approaches. While LLM+P and LATS focus on improving structured planning and decision-making, HippoRAG tackles knowledge

integration over time, addressing a different but complementary challenge in LLM-based reasoning.

Cognitive Architectures for Language Agents [6]

CoALA is a framework that systematically organizes language agents by drawing from cognitive science and symbolic AI. CoALA models language agents with modular memory components, structured action spaces, and generalized decision-making, allowing them to interact more effectively with both internal memory and external environments. This work provides a theoretical foundation for understanding and advancing LLM-based agents. CoALA offers a higher-level architectural perspective that contextualizes these approaches within broader AI and cognitive science traditions. CoALA provides a roadmap for integrating advanced reasoning, planning, and memory mechanisms—potentially guiding future improvements in both LLM+P’s structured planning capabilities and LATS’s adaptive decision-making strategies.

Eyes Wide Shut? Exploring the Visual Shortcomings of Multimodal LLMs [7]

This paper investigates the limitations of vision-language models, particularly their reliance on CLIP-based visual representations, which often misinterpret visual differences. The paper introduces the Multimodal Visual Patterns benchmark to systematically analyze these failures and proposes a Mixture of Features approach to improve visual grounding. While LLM+P and LATS focus on improving structured planning and adaptive decision-making in language-based reasoning, this work highlights a critical challenge in multimodal decision-making, where incorrect visual interpretations could lead to poor downstream reasoning and planning in multimodal agents. Addressing these visual shortcomings could significantly enhance LLM agents’ ability to integrate vision with structured planning (LLM+P) and adaptive reasoning (LATS), enabling more reliable decision-making in real-world multimodal environments.

VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks [8]

This paper extends the evaluation of LLM-based autonomous agents to realistic, visually grounded web interactions. While prior benchmarks have primarily assessed text-based reasoning and decision-making, this work highlights the importance of processing both textual and visual information for web-based automation. The study reveals key limitations in multimodal agents, particularly in integrating vision with planning, which are essential for tasks such as web navigation and user interaction. This connects directly to LLM+P and LATS, as both focus on improving structured planning and adaptive decision-making, but largely in text-based settings.

Solution Overview

LLM+P: Empowering Large Language Models with Optimal Planning Proficiency [3]

LLM+P is an agent for solving robot planning problems. Given a formally-defined domain (set of states, actions, and transitions) and a specific problem (initial state and goal states) in Planning Domain Definition Language (PDDL), existing planners can efficiently generate an

optimal sequence of actions. LLM+P essentially performs machine translation to convert a natural language description of a planning problem to PDDL—allowing the planner to directly solve the problem—and convert the generated output back to natural language (or give it to the robot).

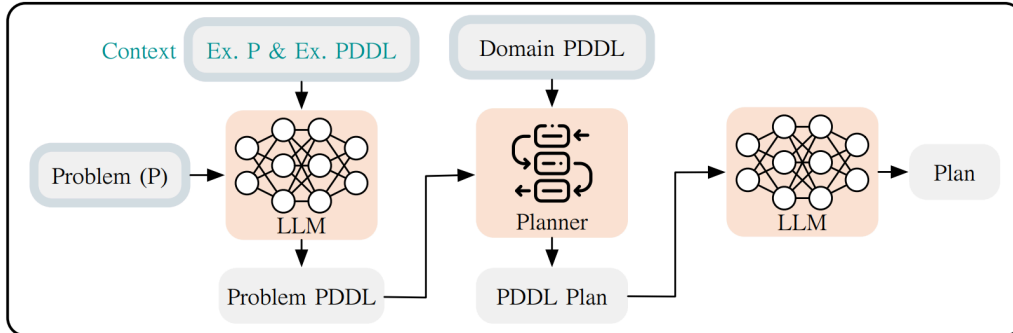


Figure 1: LLM+P’s method for solving a robot planning problem.

The intuition is that there already exist tools that can solve problems in specific domains that LLMs struggle with, such as calculators for arithmetic problems, so it is not necessary to train the LLM to be able solve every type of problem by itself. This approach leverages the main strengths of the LLM (linguistic ability) and the existing tool (functional ability). The paper’s key insight is to provide context to the LLM in the form of a simple example problem description in natural language along with its PDDL translation. This was crucial for LLM+P to solve planning problems, as seen by comparing the last two columns in table 1.

Results:

Domain	Success Rate %				
	LLM ⁻	LLM	LLM ^{ToT}	LLM+P ⁻	LLM+P
BARMAN	0	0	0	0	20 (100)
BLOCKSWORLD	20	15 (30)	0 (5)	0	90
FLOORTILE	0	0	0	0	0
GRIPPERS	25 (60)	35 (50)	10 (20)	0	95 (100)
STORAGE	0	0 (25)	0	0	85
TERMES	0	0	0	0	20
TYREWORLD	5	15	0	0	10 (90)

Table 1: Result of attempting various robot planning problems with (LLM+P) and without (LLM) using a planner. The (-) superscript indicates no context was provided to the LLM.

Using the LLM directly to solve planning problems fails with or without being given an example plan as context, especially for problems that have complex spatial relationships. Meanwhile, LLM+P successfully produces optimal plans for most robot planning problems if given context.

AutoGPT+P: Affordance-based Task Planning with Large Language Models [10]

To address the limitations of LLM+P—namely that it is restricted to closed-world settings and needs formal definitions for objects and actions—this paper proposes a more flexible way to

leverage existing tools to solve robot planning problems.

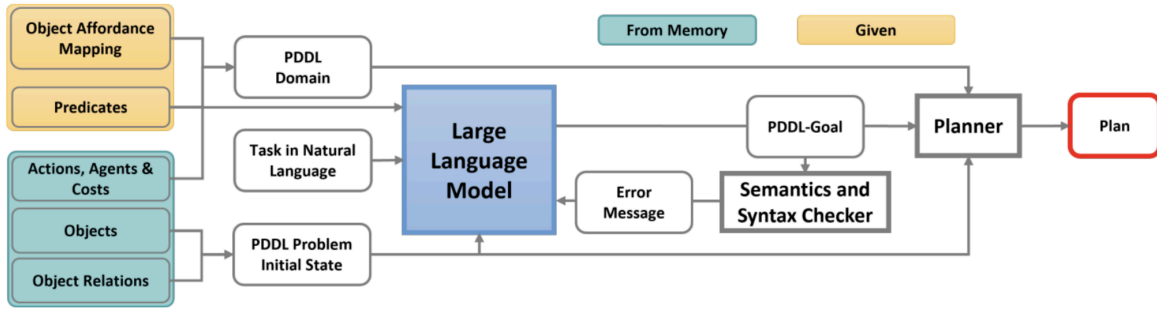


Figure 2: AutoGPT+P's method for solving a robot planning problem.

AutoGPT+P automatically detects objects on its own, allowing it to explore an open-ended setting. A key detail is using affordance to provide information about possible actions, which is implemented by mapping each object to the possible actions that they afford using ChatGPT. The agent continuously evaluates whether it has the necessary affordances to complete the task, allowing it to seek human assistance if the task is above the agent's capabilities. As shown in figure 2, the agent also performs automatic error correction on its machine translations, another improvement over LLM+P.

Results:

Instruction Family	SayCan (plan)	GPT-3 As Planner	GPT-3 As Planner+A	GPT-4 As Planner	GPT-4 As Planner+A	GPT-3 Ours	GPT-3 Ours (Auto)	GPT-4 Ours	GPT-4 Ours (Auto)
NL Primitive	0.93	0.47	0.53	0.93	0.93	0.73	0.73	1.00	1.00
NL Verb	0.60	0.00	0.00	0.67	0.87	0.27	0.33	0.93	1.00
NL Noun	0.93	0.13	0.07	0.26	0.20	0.27	0.40	0.93	1.00
Structured	0.93	0.20	0.13	0.87	0.60	0.00	0.13	0.20	0.93
Embodiment	0.64	0.09	0.00	0.55	0.55	0.64	0.64	0.82	1.00
Crowd-Sourced	0.73	0.13	0.07	0.93	0.73	0.27	0.33	0.73	0.93
Long-Horizon	0.73	0.00	0.00	0.40	0.33	0.20	0.33	0.80	1.00
Drawer	1.00	0.00	0.00	0.00	0.00	0.66	0.33	1.00	1.00
Average	0.81	0.14	0.12	0.66	0.59	0.34	0.40	0.78	0.98

Table 2: Result of AutoGPT+P generating plans for various types of tasks compared to other task planning agents.

AutoGPT+P consistently had a perfect planning success rate in translating user-specified tasks into valid plans for various types of instructions. The results show the potential of AutoGPT+P in completing real-world tasks, especially for scenarios without well-defined objects and actions.

Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models [4]

The paper proposes Language Agent Tree Search (LATS), which incorporates Monte Carlo Tree Search, environmental feedback, and LM-powered value functions for better

exploration/planning. LATS integrates the three core capabilities in one framework: (1) **Reasoning**: generating internal chains-of-thought (2) **Planning**: Employs a Monte Carlo Tree Search for exploring alternatives (3) **Acting**: Interacting with external environments like API calls and web navigation. The input is given in the form of a natural language query for the Language Model. The model samples multiple candidate reasoning and action trajectories. External feedback is received from an environment (e.g. test suite results, API responses), and then self-reflection is applied to failed trajectories to guide future searches.

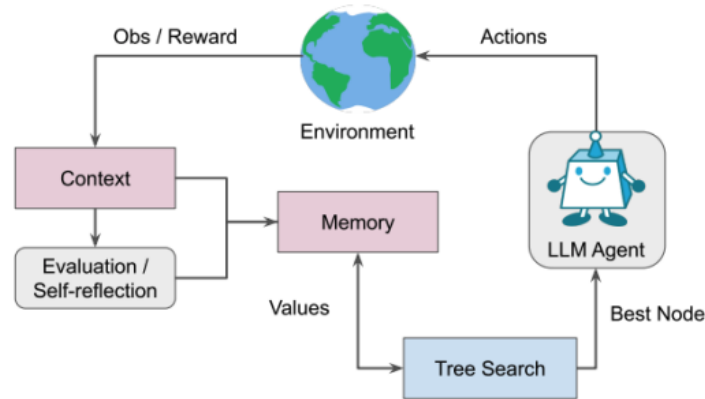


Figure 3: Overview of LATS. Serving as a unified framework, LATS leverages an external framework and MCTS-based search algorithm to improve reasoning and decision-making.

The different steps in the LATS method involve the following:

1. **Selection**: This identifies the segment of the current tree most suitable for subsequent expansion. We start from the root node (s_0), and select children nodes until we reach the leaf node.
2. **Expansion**: Next up, we expand the tree by sampling n actions from the language model (p_x). Environment variables receive action and return feedback as observation. We get n new child nodes added to the tree and then store the tree in an external long-term memory structure.
3. **Evaluation**: Next, we assign a scalar value to each child for selection and backpropagation. LATS does not involve training a novel value function based on a self-generated LM score, or a self-consistency score.
4. **Simulation**: Repeat expansion and evaluation and prioritize the highest value nodes. The terminal state gives objective feedback on correctness. Success is determined by the environment conditions.
5. **Backpropagation**: Update the values $V(s)$ of the tree based on trajectory outcome.
For each node, update the value based on $N(s_i) = N(s_{i-1}) + 1$ and $V(s_i) = \frac{V(s_{i-1})N(s_{i-1}) + r}{N(s_i)}$
6. **Reflection**: When an unsuccessful terminal node is reached, we prompt the LLM with the trajectory and final reward to self-reflect by summarizing errors in reasoning. We then store failed trajectories and reflections in the memory component

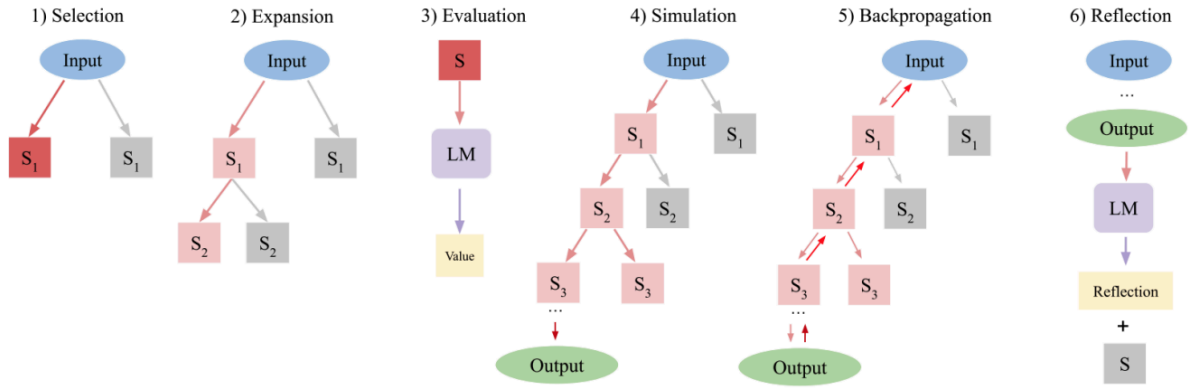


Figure 4: The LATS method and each of the different steps are outlined.

The paper set up experiments to see how LATS performs in different scenarios:

1. HotPotQA: multi-hop question-answering benchmark requiring retrieval of 2+ Wikipedia passages
2. Programming: HumanEval, MBPP(Mostly Basic Python Problems)
3. WebShop: Online shopping environment with 1.18M real-world products and 12k human instructions.
4. Game of 24: Construct 24 out of a set of numbers and basic operations.

Results:

Prompt Method	HotpotQA (EM) ↑	Prompt Method	HotpotQA (EM) ↑
Base LM	0.32	ReAct (Yao et al., 2023b)	0.32
CoT (Wei et al., 2022)	0.34	ReAct (best of k)	0.38
CoT - SC (Wang et al., 2022)	0.38	Reflexion (Shinn et al., 2023)	0.51
ToT (Yao et al., 2023a)	0.55	ToT (ReAct)	0.39
RAP (Hao et al., 2023)	0.60	RAP (ReAct)	0.54
RAP ($n = 10$)	0.60	LATS (ReAct)	0.63
LATS (CoT)	0.62	LATS ($n = 3$)	0.58
		LATS ($n = 10$)	0.65
		LATS (CoT + ReAct)	0.71

Table 3: Comparison of different baselines on HotpotQA.

As shown in Table 3, LATS outperforms all baselines on reasoning and action for the Hotpot QA. From Tables 4 and 5, we see that LATS beats all baselines for normal Chain-of-Thought, as well as for normal ReAct.

Prompt Method	Model	Pass@1 ↑
CoT (Wei et al., 2022)	GPT-3.5	46.9
ReAct (Yao et al., 2023b)	GPT-3.5	56.9
Reflexion (Shinn et al., 2023)	GPT-3.5	68.1
ToT (Yao et al., 2023a)	GPT-3.5	54.4
RAP (Hao et al., 2023)	GPT-3.5	63.1
LATS (ReAct)	GPT-3.5	83.8
Base LM	GPT-4	80.1
Reflexion	GPT-4	91.0
LATS (ReAct)	GPT-4	92.7

Prompt Method	Pass@1 ↑
CoT (Wei et al., 2022)	54.9
ReAct (Wei et al., 2022)	67.0
Reflexion (Shinn et al., 2023)	70.0
ToT (Yao et al., 2023a)	65.8
RAP (Hao et al., 2023)	71.4
LATS (ReAct)	81.1

(Left) Table 4: GPT 3.5 and GPT 4.0 Pass@1 accuracy on HumanEval. Prompting with LATS achieves the best performance. (Right) Table 5: GPT-3.5 Pass@1 accuracy on MBPP. Prompting with LATS achieves the highest performance.

In the Webshop scenario results in Table 6, LATS even beats the fine-tuning model and gets close to the expert performance as well. In that scenario, LATS does not beat the success rate (SR) but does have a better score. For the game of 24 results shown below in Table 7, they construct 24 out of a set of numbers and basic operations and here LATS beats the competition.

Method	Score ↑	SR ↑
ReAct (Yao et al., 2023b)	53.8	28.0
ReAct (best of k)	59.1	32.0
Reflexion (Shinn et al., 2023)	64.2	35.0
LATS (ReAct)	75.9	38.0
IL (Yao et al., 2022)	59.9	29.1
IL+RL (Yao et al., 2022)	62.4	28.7
Fine-tuning (Furuta et al., 2024)	67.5	45.0
Expert	82.1	59.6

Prompt Method	Game of 24 (Success Rate) ↑
CoT (Wei et al., 2022)	0.08
Reflexion (Shinn et al., 2023)	0.12
ToT (Yao et al., 2023a)	0.20
RAP (Hao et al., 2023)	0.40
LATS (CoT)	0.44

(Left) Table 6: Score and success rate (SR) on WebShop. Results were organized into prompting, RL-based training, and human performance. (Right) Table 7: Results on Game of 24 with GPT-3.5. Sampled $n = 5$ nodes and $k = 30$ trajectories.

TravelPlanner: A Benchmark for Real-World Planning with Language Agents [9]

In order to evaluate the planning capabilities of LLMs, the authors propose a new planning benchmark, TravelPlanner, which consists of approximately 4 million records of real-world travel data, including flights, hotels, and activities. The benchmark requires models to generate itineraries that satisfy multiple hard constraints, such as ensuring that flights do not overlap, hotels are within budget, and activities fit into a logical daily schedule. Various LLM-based planning techniques are evaluated, such as tool-augmented agents (which leverage APIs for real-time trip data), direct prompting, and iterative refinement procedures, which allow models to make changes to their plans in response to error input. Human annotators are used for qualitative review, while the evaluation framework automatically looks for feasibility breaches (such as impractical layovers or exceeding budgetary

restrictions).

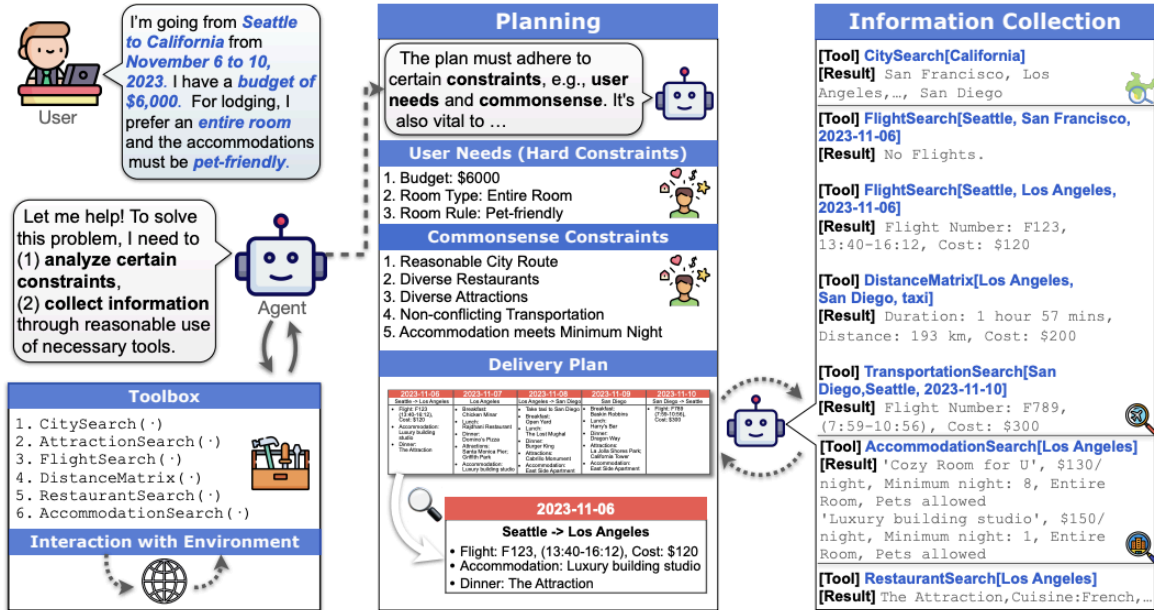


Figure 5: Overview of TravelPlanner. Given a query, language agents are tasked with employing various search tools to gather information. Based on the collected information, language agents are expected to deliver a plan that not only satisfies the user's needs specified in the query but also adheres to commonsense constraints.

To improve performance, TravelPlanner also incorporates external tool usage, testing whether LLMs can successfully call APIs (e.g., Google Flights) to retrieve real-time availability and pricing information. Models that can dynamically integrate external data perform significantly better than those relying purely on static knowledge. Additionally, the benchmark assesses iterative planning, where agents can detect constraint violations and refine their itineraries accordingly. This iterative approach helps mitigate common LLM failures, such as generating overlapping flight schedules or unrealistic travel sequences.

Tool-use	Direct	Reflexion
Query: Please plan a travel itinerary for me. I'm departing from Cincinnati and heading to Norfolk for three days. The dates of travel are from March 10th to March 12th, 2022. I have a budget of \$1,400 for this trip. Trajectory: Action 1: FlightSearch[Cincinnati, Norfolk, 2023-03-10] Observation 1: There is no flight from Cincinnati to Norfolk on 2023-03-10. Action 2: FlightSearch[Cincinnati, Norfolk, 2023-03-09] Action 3: FlightSearch[Cincinnati, Norfolk, 2023-03-11] Action 4: FlightSearch[Cincinnati, Richmond, 2023-03-10] ... Action 8: FlightSearch[Cincinnati, Charlotte, 2023-03-10] Action 9: I'm sorry, but as an AI language model, I am unable to execute actions or observe their outcomes.	Query: Please create a travel itinerary for a solo traveler departing from Jacksonville and heading to Los Angeles for a period of 3 days, from March 25th to March 27th, 2022. The budget for this trip is now set at \$2,400. Plan: Day 1: transportation: Flight Number: F3765485, from Jacksonville to Los Angeles ... accommodation: Midtown East 2 Bed/2 Bath Elevator Building Apt (7-day minimum night) Day 2: ... accommodation: Midtown East 2 Bed/2 Bath Elevator Building Apt (7-day minimum night) Day 3: transportation: Flight Number: F3765485, from Los Angeles to Jacksonville ...	Query: Can you help construct a travel plan that begins in Philadelphia and includes visits to 3 cities in Virginia? The trip duration is for 7 days, from March 15th to March 21st, 2022, with a total budget of \$1,800. Trajectory: ... Observation 3: The cost of your current plan is 316.0 dollars. Thought 4: I need to select a more budget-friendly accommodation to keep the overall trip within the \$1,800 budget. Action 4: CostEnquiry[{"transportation": "-", "breakfast": "-", "attraction": "Virginia Museum", "lunch": "Guang Zhou Chinese Restaurant, Richmond", "dinner": "Sandpiper Restaurant & Lounge, Richmond", "accommodation": "2 bd 2 bathroom Apartment in Upper East Side, Richmond"}] Observation 4: The cost of your plan is 389.0 dollars.

Figure 6: Case studies of failures. Agents fail to finalize plans due to repeated errors like incorrect dates, confusion with information

details leading to hallucinatory answers, and disconnects between reasoning and actions. All cases are gained from GPT-4-Turbo based

agents.

Results:

According to TravelPlanner's evaluation of several cutting-edge LLMs, most have difficulty adhering to limitations, particularly when managing complex itineraries with multiple phases. LLMs benefit significantly from accessing real-world data, as demonstrated by the superior performance of models that incorporate external tool augmentation (such as retrieving real-time flight data). However, even the top-performing models fall short on a substantial portion of tasks and often violate feasibility guidelines (e.g., booking an event before arrival). Iterative feedback models improve constraint satisfaction rates, although they do not match human-level planning.

	Validation (#180)						Test (#1,000)					
	Delivery Rate	Commonsense Pass Rate		Hard Constraint Pass Rate		Final Pass Rate	Delivery Rate	Commonsense Pass Rate		Hard Constraint Pass Rate		Final Pass Rate
		Micro	Macro	Micro	Macro			Micro	Macro	Micro	Macro	
Greedy Search	100	74.4	0	60.8	37.8	0	100	72.0	0	52.4	31.8	0
Two-stage												
Mistral-7B-32K (Jiang et al., 2023)	8.9	5.9	0	0	0	0	7.0	4.8	0	0	0	0
Mixtral-8x7B-MoE (Jiang et al., 2024)	49.4	30.0	0	1.2	0.6	0	51.2	32.2	0.2	0.7	0.4	0
Gemini Pro (G Team et al., 2023)	28.9	18.9	0	0.5	0.6	0	39.1	24.9	0	0.6	0.1	0
GPT-3.5-Turbo (OpenAI, 2022)	86.7	54.0	0	0	0	0	91.8	57.9	0	0.5	0.6	0
GPT-4-Turbo (OpenAI, 2023)	89.4	61.1	2.8	15.2	10.6	0.6	93.1	63.3	2.0	10.5	5.5	0.6
Sole-planning												
DirectGPT-3.5-Turbo	100	60.2	4.4	11.0	2.8	0	100	59.5	2.7	9.5	4.4	0.6
CoTGPT-3.5-Turbo	100	66.3	3.3	11.9	5.0	0	100	64.4	2.3	9.8	3.8	0.4
ReActGPT-3.5-Turbo	82.2	47.6	3.9	11.4	6.7	0.6	81.6	45.9	2.5	10.7	3.1	0.7
ReflexionGPT-3.5-Turbo	93.9	53.8	2.8	11.0	2.8	0	92.1	52.1	2.2	9.9	3.8	0.6
DirectMixtral-8x7B-MoE	100	68.1	5.0	3.3	1.1	0	99.3	67.0	3.7	3.9	1.6	0.7
DirectGemini Pro	93.9	65.0	8.3	9.3	4.4	0.6	93.7	64.7	7.9	10.6	4.7	2.1
DirectGPT-4-Turbo	100	80.4	17.2	47.1	22.2	4.4	100	80.6	15.2	44.3	23.1	4.4

Table 8: Main results of different LLMs and planning strategies on the TravelPlanner validation and test set. The best results are marked in bold. When the collected information is insufficient, Gemini Pro tends to directly refuse to provide the plan. Interviews with annotators reveal that manually annotating a plan averagely takes around 12 minutes. However, language agents, such as GPT-3.5-Turbo, can accomplish this task in just 1 to 2 minutes, showcasing their efficiency.

Limitations

LLM+P

1. It assumes the agent recognizes when to trigger the LLM+P pipeline during a conversation with the user.
2. The problem domain (formal object/action declarations) must be predefined in PDDL and provided to the agent (restricting it to closed-world problems).
3. A simple problem description in natural language and its corresponding PDDL

translation also must be provided as context to the agent.

AutoGPT+P

1. There is no feedback from the executed plan.
2. It is unclear how well the agent performs on larger scale tasks (the paper did not consider plans longer than 20 steps).

LATS

1. Higher Computational Costs than simpler methods like ReAct/Reflexion
2. Assumes reversible states in the environment - not universally acceptable

Travel Planner

1. LLMs struggle with incorrect tool use and dead loops.
2. As the number of constraints increase, the performance of LLMs drops.
3. The TravelPlanner benchmark favors rigid adherence to predefined plans, limiting flexibility.

Future Research Directions

1. Multi-Agent LATS and utilizing richer external feedback from multimodal environments.
2. Enabling LLM+P (and other LLM+Tool agents) to auto-detect when to use the tool.
3. Planner agents that can handle larger amounts of constraints and information.
4. Developing LLMs that can dynamically refine plans based on real-time hard constraints.
5. Incorporating user feedback loops for improved itinerary generation for travel planning.

Summary of Class Discussion

We list a few questions that are most interesting and widely discussed in the class.

Q1: [LLM+P] For LLM+P to work, it requires feeding the LLM the context of the specific tasks. Can it be generalized to other tasks?

A1: No, you would have to define the context for each task. This is one of the big restrictions of LLM+P when compared to the other related works. LLM+P works well when you have a predefined problem with formal definitions, otherwise it is very limited.

Q2: [LATS] While LATS improves performance in long-horizon tasks, it operates at the cost of additional computational overhead. How does its efficiency compare to simpler prompting strategies like Chain-of-Thought or self-consistency, and in what scenarios is the extra compute justified?

A2: It is a lot more computationally expensive as compared to Chain-Of-Thought because instead of utilizing a one-shot approach to Chain-Of-Thought, LATS uses evaluation and reasoning at each step to evaluate the chain-of-thought framework interrupted, and then continues it. The best scenario where this would be useful is long-horizon multiple-objective tasks where the additional compute is justifiable like the Travel Planning scenario.

Q3: **[LATS]** LATS was tested on diverse domains such as programming and online shopping. Did the authors have to do any manual adjustments to the agent in between each task or does it automatically work on all them? For example, in the programming task each state could be considered a final state, removing the need for the expansion/simulation step.

A3: In general, for LATS you do need to try to fit the problem into the framework, whereas ideally the framework can fit into any problem. For the programming example, while each state could be a final answer, it is also possible that each state aims to pass a smaller subset of the test suite, so there actually would be an expansion step as each piece of code builds off the previous piece.

Q4: **[LATS]** In robotics, financial trading, or emergency response systems, once an action is taken, the environment changes permanently, making past states inaccessible. Given this limitation, how does LATS handle such scenarios? Would modifications be necessary to adapt it for irreversible decision-making tasks, and if so, what changes would make it more applicable to real-world use cases?

A4: In financial trading, once a trade is executed, it's final, and market conditions continuously evolve. To adapt LATS, it could incorporate real-time market data and probabilistic forecasting to make decisions based on expected outcomes rather than backtracking. In emergency response, actions like dispatching ambulances or issuing evacuation orders cannot be undone. Instead of tree search, a reinforcement learning approach trained on past emergency scenarios could help the system make optimal decisions in real time. Both cases require shifting LATS from a reversible search framework to a forward-planning system that prioritizes adaptability and risk-aware decision-making.

Q5: **[TravelPlanner]** How can we circumvent hallucination without using brute-force?

A5: There were not any relevant findings from the paper, however it could be interesting to use a tree structure such as the one used in LATS and use a heuristic for picking the best action from multiple options.

References

1. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *ArXiv*.
2. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. *ArXiv*.
3. Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., & Stone, P. (2023). LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. *ArXiv*.
4. Zhou, A., Yan, K., Wang, H., & Wang, Y. (2023). Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *ArXiv*.
5. Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. *ArXiv*.
6. Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2023). Cognitive

Architectures for Language Agents. *ArXiv*.

7. Tong, S., Liu, Z., Zhai, Y., Ma, Y., LeCun, Y., & Xie, S. (2024). Eyes Wide Shut? Exploring the Visual Shortcomings of Multimodal LLMs. *ArXiv*.

8. Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P., Neubig, G., Zhou, S., Salakhutdinov, R., & Fried, D. (2024). VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. *ArXiv*.

9. Xie, J., Zhang, K., Chen, J., Zhu, T., Lou, R., Tian, Y., Xiao, Y., & Su, Y. (2024). TravelPlanner: A Benchmark for Real-World Planning with Language Agents. *ArXiv*.

10. Birr, Timo et al. "AutoGPT+P: Affordance-based Task Planning with Large Language Models." *ArXiv* abs/2402.10778 (2024): n. pag.